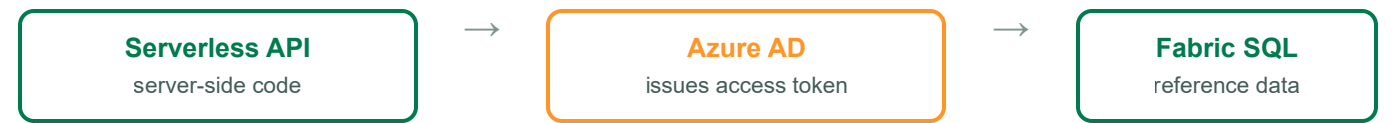


Azure / Fabric SQL Connection Guide

How the app connects to Microsoft Fabric SQL for reference data via an Azure AD service principal — and how to set up the same connection on a new project.

1. How the connection works

Reference data (teams, clients, chains, stores, items) is read live from a **Microsoft Fabric SQL analytics endpoint**. The connection is authenticated with an **Azure AD service principal** (an app registration) — there is no SQL username/password. The app requests a short-lived Azure AD *access token* and connects with it. Only the server-side code holds credentials; the browser never connects to SQL.



Step by step, inside the app

1. Acquire an Azure AD token for scope `https://database.windows.net/.default`.
2. In production, use `ClientSecretCredential` (tenant id + client id + client secret). Locally, if those are absent, fall back to `DefaultAzureCredential` — i.e. your `az login` session.
3. Open a `tedious` (TDS) connection to the Fabric SQL server on port **1433**, `encrypt: true`, authenticating with the token (`azure-active-directory-access-token`).
4. Run parameterized queries; transient failures (timeouts, dropped sockets, token issues) are retried automatically.

Libraries used: `@azure/identity` (token acquisition) and `tedious` (SQL Server / Fabric TDS driver).

2. Environment variables

These are the only values needed for the Azure / Fabric connection. Store them as environment variables — in the hosting platform (e.g. Vercel → Settings → Environment Variables) for production, or in a git-ignored `.env.local` for local development. Never commit them.

VARIABLE	PURPOSE	WHERE IT COMES FROM
AAD_TENANT_ID	Azure AD tenant that owns the service principal	Azure Portal → app registration → Overview
AAD_CLIENT_ID	Service principal (application) client id	Azure Portal → app registration → Overview
AAD_CLIENT_SECRET	Service principal client secret	App registration → Certificates & secrets

FABRIC_SQL_SERVER	Fabric SQL endpoint host name	Fabric → SQL analytics endpoint → Connection string
-------------------	-------------------------------	---

FABRIC_SQL_DATABASE	Database / lakehouse name	Fabric SQL analytics endpoint
---------------------	---------------------------	-------------------------------

If `AAD_CLIENT_ID` / `AAD_CLIENT_SECRET` are not set, the code falls back to `DefaultAzureCredential` (useful locally with `az login`). If `FABRIC_SQL_SERVER` is not set, reference endpoints return `503` cleanly instead of failing.

3. Setting up the connection on a new project

1. **Register (or reuse) the Azure AD service principal.** In Azure Portal → App registrations, reuse the existing "WorkFlag Submission" registration or create a new one. Record the **Tenant ID** and **Client ID** from the Overview page.
2. **Create a client secret.** App registration → Certificates & secrets → New client secret. Copy the value immediately (it is shown only once) into `AAD_CLIENT_SECRET`.
3. **Grant the principal access in Fabric.** Open the Fabric workspace that exposes the SQL analytics endpoint → **Manage access** → add the service principal with an appropriate role (Viewer/Contributor). See the gotcha below.
4. **Get the Fabric connection details.** In Fabric, open the SQL analytics endpoint → Settings / Connection string. Copy the **server** host into `FABRIC_SQL_SERVER` and the database/lakehouse name into `FABRIC_SQL_DATABASE`.
5. **Set the five environment variables** (section 2) in the new project's settings.
6. **Deploy and verify** (section 5).

Key gotcha — grant access in Fabric, not with SQL. The Fabric endpoint is a Lakehouse/Warehouse SQL analytics endpoint, so `CREATE USER ... FROM EXTERNAL PROVIDER` fails with **Msg 22424**. Add the service principal through the Fabric workspace **Manage access** UI instead — that is what grants the token permission to read the endpoint.

4. Local development

Two options for authenticating locally:

- **Service principal:** put `AAD_TENANT_ID`, `AAD_CLIENT_ID`, `AAD_CLIENT_SECRET`, and the `FABRIC_SQL_*` values in a git-ignored `.env.local`.
- **Your own Azure identity:** run `az login` and set only the `FABRIC_SQL_*` values — the code uses `DefaultAzureCredential` automatically.

```
cp .env.local.example .env.local # then fill in the AAD_* and FABRIC_SQL_* values
# ...or, to use your own identity:
az login
```

5. Verifying the connection

- Load a screen that lists teams / clients / chains. If the lists populate, the token and Fabric grant are working.
- **Empty lists** usually mean the service principal is not granted in the Fabric workspace, or a team/period filter mismatch. Check the server logs for an Azure AD token error or a login failure.
- A message like *"Could not login because the authentication failed"* points to a missing Fabric access grant (step 3) or an expired/incorrect client secret.
- Always test against the **latest deployment URL** — a stale pinned preview URL will not have your newest environment variables.

6. Security notes

- Never commit secrets. `.env.local` is git-ignored; paste values directly into the hosting platform's environment fields.
- Treat `AAD_CLIENT_SECRET` as highly sensitive. Rotate it on a schedule and whenever staff with access change.
- The browser never sees the token or the secret — only the server-side code requests the token and opens the SQL connection.
- When running a local server, bind to `127.0.0.1` rather than `0.0.0.0`.

Advantage Solutions · HQ to Retail Connector — Azure / Fabric SQL connection reference. Auth: Azure AD service principal → access token → Fabric SQL (read-only reference data).